Bachelor Thesis

Quantum network simulator for quantum key distribution

by

Gonzalo Vela

(gve218)

First supervisor: Wan Fokkink

June 8, 2026

Submitted in partial fulfillment of the requirements for the VU degree of Bachelor of Science

Quantum network simulator for quantum key distribution

Gonzalo Vela
Vrije Universiteit Amsterdam
Amsterdam, The Netherlands
g.vela@student.vu.nl

ABSTRACT

Context. Write this at the end
Goal. Write this at the end
Method. Write this at the end
Results. Write this at the end
Conclusions. Write this at the end

1 INTRODUCTION

The secure transmission of sensitive data is one of the key challenges of the modern digital era. Classical cryptographic algorithms such as RSA [7] face a significant threat from the emergence of quantum computers, as their security relies on the assumption that no efficient algorithm exists to invert the encryption using publicly available information. Quantum Key Distribution (QKD) has emerged as a provably secure alternative, grounded in the laws of quantum physics rather than computational complexity.

Many QKD protocols exist [1, 3, 10] that enable the generation of secure keys by exploiting the principles of quantum mechanics. This thesis focuses on implementing the E91 protocol due to its use of entangled qubits and Bell-state verification.

Transitioning these theoretical protocols into scalable real-world networks presents significant engineering challenges. For this reason, quantum network simulators such as SeQUeNCe [12] and QKDNetSim [6] are valuable tools, as they enable controlled testing of new protocols. Nevertheless, the development of such simulators introduces additional challenges, including the accurate modeling of entanglement and distance-dependent fidelity degradation. Due to these limitations and the lack of consensus among existing models, several assumptions were made, which are discussed later in this thesis.

This thesis addresses the following research question: *How can we design and implement an event-driven quantum network simulator capable of evaluating the performance, error-correction overhead, and adversarial resilience of the E91 protocol under realistic hardware imperfections?*

To answer this question, we present the design and implementation of a modular, high-performance quantum network simulator written in Rust [4]. Rust was chosen for its memory safety, high performance, and concurrency support without the overhead of a garbage collector. The core architecture is based on a discrete event-driven simulation engine that uses a priority queue to process events at picosecond resolution (*ps*). The platform models realistic quantum channels, including distance-dependent entanglement degradation, optical fiber attenuation, and detector cooldown states. It also implements a complete classical post-processing pipeline, including basis reconciliation, CHSH inequality testing, and Cascade error correction. Furthermore, a reactive web interface built with Leptos and compiled to WebAssembly provides interactive real-time visualization of network topologies.

Through targeted simulation scenarios, we evaluate the performance of the E91 protocol over distances of up to approximately 150 km, assuming a 0.14% fidelity degradation per km based on [8]. To validate the security subsystem, we introduce an active eavesdropper model ("Mallory") performing interception attacks on transmitted qubits. The simulator demonstrates that such interference destroys quantum correlations, reducing the CHSH parameter S below the classical bound ($S < 2$) and triggering protocol termination.

The main contributions of this thesis are as follows:

Event-Driven Simulation Engine: We design and implement a high-performance discrete-event simulation engine in Rust using a priority queue. The engine resolves events at picosecond resolution, accurately modeling asynchronous node behavior, qubit propagation delays, and detector cooldown dynamics.

Realistic Quantum Channel and Device Modeling: We implement detailed abstractions of optical channels and quantum devices, including distance-dependent fidelity degradation, detector cooldwn, and Poisson-distributed dark counts.

End-to-End E91 Post-Processing and Security Verification: We construct a complete pipeline for key sifting, CHSH inequality testing, and multi-round Cascade error correction. The simulator reliably detects eavesdropping attempts through degradation of the Bell parameter ($S < 2$).

The remainder of this thesis is organized as follows: Section 2 introduces quantum entanglement, the E91 protocol, and the Cascade error correction algorithm. Section 3 describes the simulator architecture and development challenges. Section 4 presents simulation results and their relevance to real-world scenarios. Finally, Section 7 compares the proposed simulator with existing tools before the concluding section.

2 BACKGROUND

Before discussing the design and evaluation of the simulator, this section introduces the fundamental quantum concepts underlying the E91 key distribution protocol.

In E91, as in any quantum communication system, information is transmitted using **quantum states**. These states encode probability amplitudes whose squared magnitudes determine the likelihood of measurement outcomes. A quantum state that is expressed as a linear combination of basis states is said to be in a **superposition**.

The simplest representation of a quantum system is the **qubit**, a two-level quantum state defined in the computational basis as:

$$|0\rangle\,, \quad |1\rangle\,.$$

Using Born's rule, the state of a qubit can be expressed as a linear combination of these basis states:

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle\,,$$

where $\alpha, \beta \in \mathbb{C}$ and satisfy the probability condition $|\alpha|^2 + |\beta|^2 = 1$.

To operate on qubits, we can use a set of quantum gates analogous to logic gates in classical computing. One example is the **Hadamard (H)** gate, which creates superposition states and is represented by the following matrix:

$$H = \frac{1}{\sqrt{2}}\begin{pmatrix}1 & 1\\ 1 & -1\end{pmatrix}.$$

For multi-qubit systems, one of the most important gates is the **CNOT** (controlled-NOT) gate, which acts on a two-qubit system. Its matrix representation is given by:

$$\text{CNOT} = \begin{pmatrix}1 & 0 & 0 & 0\\ 0 & 1 & 0 & 0\\ 0 & 0 & 0 & 1\\ 0 & 0 & 1 & 0\end{pmatrix}.$$

Quantum states can also be represented as vectors in a complex Hilbert space. In this representation, the application of a quantum gate corresponds to matrix multiplication between the gate operator and the state vector.

In order to transform a qubit into a superposition state, where both measurement outcomes have equal probability, the Hadamard gate is applied to a qubit initially prepared in a computational basis state, such as $|0\rangle$.

Representing the basis state $|0\rangle$ as a vector:

$$|0\rangle = \begin{pmatrix}1\\ 0\end{pmatrix},$$

we apply the Hadamard gate via matrix multiplication:

$$H\,|0\rangle = \frac{1}{\sqrt{2}}\begin{pmatrix}1 & 1\\ 1 & -1\end{pmatrix}\begin{pmatrix}1\\ 0\end{pmatrix} = \frac{1}{\sqrt{2}}\begin{pmatrix}1\\ 1\end{pmatrix} = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle).$$

Entanglement, a key aspect of the E91 protocol, occurs when two qubits are prepared in a quantum state such that their measurement outcomes exhibit correlations, regardless of distance. These entangled two-qubit states are known as Bell states, named after physicist John Bell, who developed Bell's theorem to test the non-classical nature of such correlations.

One of the four maximally entangled two-qubit Bell states is defined as:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle),$$

In order to produce a Bell state, you first apply a Hadamard gate, followed by a CNOT gate.

When both qubits of the Bell state are measured in the same basis, the outcomes are perfectly correlated, giving identical results (either 00 or 11) despite each result being individually random. This basis-dependent correlation is the key resource exploited in the E91 protocol, enabling both key generation and eavesdropping detection through later comparison of measurement outcomes.

Building on these quantum mechanical principles, the E91 protocol enables two nodes in the quantum internet, to establish a shared secret key using entangled qubit pairs distributed by a quantum source.

E91 relies on the distribution of entangled states, typically Bell states, such that correlations between measurement outcomes can be used both for key generation and for detecting the presence of an eavesdropper.

In the standard setting, a trusted, usually refered as epr node, distributes entangled qubit pairs to both of the client nodes over a quantum channel. Each party independently measures their qubits using randomly chosen measurement bases. The choice of measurement basis plays a central role in both the generation of the key and the security analysis of the protocol.

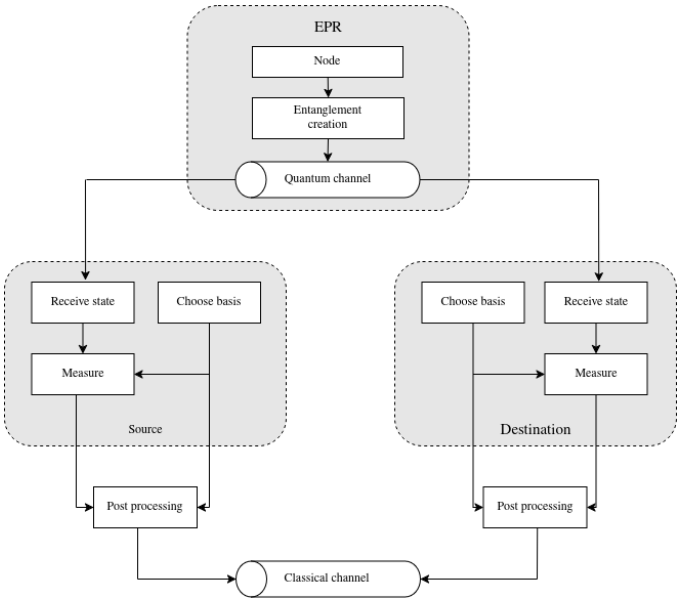

Figure 1: Diagram describing E91 process

Once all entangled pairs have been distributed and measured, both nodes share their chosen measurement bases over a classical channel, which reveals little to no useful information to an eavesdropper.

Using this basis information, the measured qubits can be divided into two groups:

- **Key qubits**: These correspond to measurements where both parties used the same basis. Assuming ideal entanglement and no fidelity loss, the resulting measurement outcomes should be perfectly correlated.
- **Test qubits**: These correspond to measurements performed in different bases. These outcomes are used to evaluate quantum correlations and to detect the presence of an eavesdropper in the communication channel.

In the E91 protocol, the measurement bases correspond to rotations on the Bloch sphere that define the detector settings used to measure incoming qubits. Following [5], the choice of measurement angles that maximizes quantum correlations and leads to a strong violation of the Bell inequality must be chosen in steps of $\frac{\pi}{8}$. The

ones chosen for this simulator were:

$$\alpha = 0 \quad (0^\circ), \qquad \alpha' = \frac{\pi}{4} \quad (45^\circ),$$
$$\beta = -\frac{\pi}{8} \quad (-22.5^\circ), \qquad \beta' = \frac{\pi}{8} \quad (22.5^\circ),$$

where α, α' correspond to one party's polarizer settings and β, β' correspond to the other party's polarizer settings.

The CHSH test is an experimental test of Bell's theorem, which shows that no physical theory based on local hidden variables can reproduce all predictions of quantum mechanics.

This result is expressed using the CHSH S-parameter [9], defined as

$$S = |E(A_0, B_0) + E(A_0, B_1)| + |E(A_1, B_0) - E(A_1, B_1)| .$$

Here, $E(A_i, B_j)$ denotes the expectation value of the correlation between measurement settings A_i and B_j.

In any local hidden variable theory, the CHSH inequality imposes the bound

$$S \leq 2,$$

whereas quantum mechanics predicts a stronger limit,

$$S \leq 2\sqrt{2}.$$

Based on [11] when the basis are used as polarizer angles (moving the detector by such an angle) the expectation value can be calculated as:

$$E(a, b) = \cos\big(2(a - b)\big)$$

Therefore, the CHSH value for our choice of basis is given by

$$E(\alpha, \beta) = \cos\left(\frac{\pi}{4}\right) = \frac{\sqrt{2}}{2}, \quad E(\alpha, \beta') = \cos\left(-\frac{\pi}{4}\right) = \frac{\sqrt{2}}{2},$$
$$E(\alpha', \beta) = \cos\left(\frac{3\pi}{4}\right) = -\frac{\sqrt{2}}{2}, \quad E(\alpha', \beta') = \cos\left(\frac{\pi}{4}\right) = \frac{\sqrt{2}}{2}.$$

Substituting into the CHSH expression,

$$S = |E(\alpha, \beta) + E(\alpha, \beta')| + |E(\alpha', \beta) - E(\alpha', \beta')| ,$$

gives

$$S = \left|\sqrt{2}\right| + \left|-\sqrt{2}\right| = 2\sqrt{2}.$$

If after calculating the S parameter, we get $|S| < 2$, we can assume that the correlation is not maximal and there is a third person in the communication

The Cascade protocol functions as a multi-pass block-parity check designed to reconcile bit errors between Alice and Bob caused by fidelity loss. In the first pass, the sifted bit strings are divided into equal-sized blocks (k_1), and Alice transmits the parity of each block to Bob. For any block where parities disagree—signaling an odd number of errors—the parties execute the BINARY subprotocol. This subprotocol bisects the block and compares the parities of the halves until the exact position of a single erroneous bit is isolated and corrected by Bob, leaving all remaining blocks with an even number of errors.

In subsequent passes ($i > 1$), the block size increases (k_i) and a new mapping function shuffles the bits into different groups. Alice and Bob exchange parities for these new blocks to identify remaining discrepancies. Correcting an error in a later pass alters the parity of the bits in earlier passes, triggering a feedback loop; the protocol identifies the smallest blocks from previous passes that now have odd errors and recursively runs BINARY to correct them. For this simulator, this framework will run for exactly four rounds of cascade to ensure identical keys before proceeding to hash-based verification.

3 DESIGN

3.1 Modularized Design

To support future adaptations and scalability, the code was structured into layers, each with defined responsibilities and behavior.

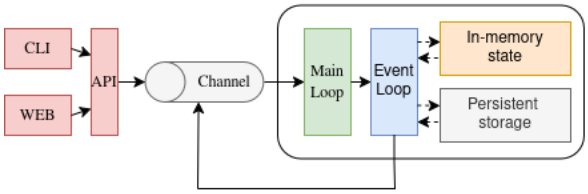

Figure 2: Diagram showing layers of the simulator

1. CLI and Web: These components serve as the primary user interfaces of the simulator. They run on separate threads and provide users with the ability to create nodes, establish links, start simulation runs, and perform many other management tasks.

The CLI operates in a loop that receives commands from the terminal and parses them accordingly. Rust enums provide type safety for commands, while the implementation of the 'From' trait enables efficient conversion between user input and the corresponding command type.

The Web interface runs on an Axum server and serves static HTML content upon request. Once initialized, the JavaScript 'Cytoscape' library enables the visualization and creation of nodes and links within the graph. HTTP is used for standard requests, while WebSockets provide fast communication during simulation runs.

2. API: This layers serves as a bridge between both the CLI/WEB and the channel. It exposes a unified set of functions that translate user requests into events executed int the simulation.

Both interfaces comply with the same function signatures and data models, ensuring consistent behavior without direct coupling between them.

3. Channel: As the simulation engine and the entry points run on different threads, a communication channel must exist between them in order for user requests to be executed by the simulation.

This channel is a single-writer single-reader channel. Instead of sharing state between both layers, which would require locking due to Rust's mutable ownership rules, the entry point threads send event requests that are received by the main loop.

4. Main Loop: This loop is responsible for both receiving requests from the channel and executing the events processed by the event loop.

The loop first checks whether there are any pending events in the channel. If so, the channel is drained until no more events remain. It then checks whether there are any events in the EventLoop; if an event is present, it executes it.

After processing the event, the loop restarts. If no events are present either in the channel or in the event loop, it blocks the

thread while waiting for new events to arrive on the channel. This is intended behavior, as new events can only be triggered by user requests and therefore appear in the channel.

5. Event Loop: This is the event loop responsible for ordering events based on the timestamp at which they must be executed.

It uses a binary heap. A binary heap is a data structure that provides $O(1)$ complexity for accessing the next event and $O(\log n)$ complexity for inserting events in the worst case.

6. In-memory State: This is the state used by the simulation for fast storage and retrieval of information during simulation runs.

It uses multiple HashMaps to store objects such as nodes, links, and measurements. A HashMap provides $O(1)$ complexity for data access and $O(1)$ complexity for data insertion.

As explained later, transitioning from fully persistent storage to an in-memory state resulted in a significant performance improvement.

7. Persistent storage: This is the storage used to persist data that must be shared across simulations. It stores information such as node and link data, generated keys, and other relevant simulation data.

3.2 Design implementations

This section explains how the different objects were simulated.

Nodes and Detectors: Nodes and detectors were implemented as independent objects; however, each node references a detector.

Each node has a `node type` attribute. The two possible types are `Client`, which is responsible for measuring qubits, and `EPR`, which is responsible for generating entangled pairs. A node is locked while it is participating in a simulation run, preventing other simulations from being executed concurrently on the same node.

Each detector has a cooldown rate. Upon reception of a qubit, if the detector is still on cooldown

```
(last_detection_time + cooldown > timestamp)
```

the qubit is dropped.

Dark count rate: The dark count rate is a constant property of the detector. As this rate increases, the average interval between dark count events decreases. Since these intervals should not be fixed but instead occur randomly in time, the process is modeled as a Poisson process. In a Poisson process, the time between consecutive events follows an exponential distribution.

As shown in Figure 3, when the parameter λ becomes smaller (which corresponds to a higher dark count rate), the distribution becomes more concentrated near zero, resulting in shorter intervals between dark count events.

Links: Links model the connection between two nodes. Each link has a `distance` attribute, which affects both the data propagation delay and the fidelity degradation over the channel.

As explained in Subsection 3.3, a constant fidelity degradation factor of 0.14% per kilometer was used. Therefore, the longer the link, the lower the entanglement fidelity of transmitted qubits.

Entanglement & Measurement: An entangled pair in a Bell state is modeled as a struct instance. This instance stores the measurements from both nodes, and the measurement logic is defined as follows:

(1) Check whether the current measurement is the first.

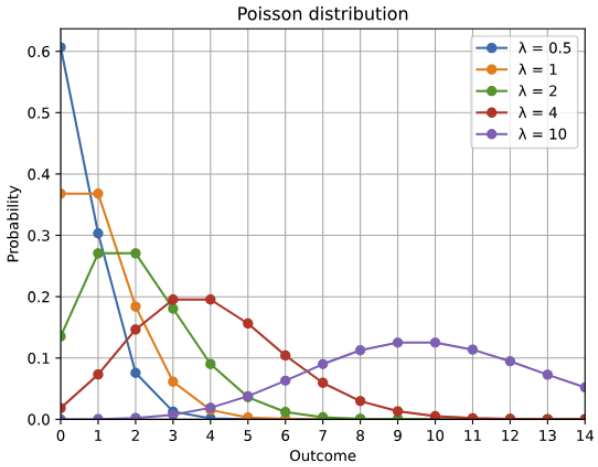

Figure 3: Diagram showing the poisson distribution

If it is the first measurement:

(a) Choose a random basis.
(b) Generate a random number between 0 and 1.
(c) Store the measurement in the corresponding measurement attribute of the pair.

If it is the second measurement:

(a) Calculate the updated fidelity after distance-based degradation.
(b) Choose a random basis.
(c) Compute the basis difference between the first measurement basis and the current one.
(d) Compute the entanglement probability.
(e) Based on this probability, either copy the first measurement outcome or take the opposite result.
(f) Store the measurement in the corresponding measurement attribute of the pair.

The entanglement probability function is as follows:

$$P_{\text{same}} = F \cdot \cos^2(\alpha - \beta) + (1 - F) \cdot 0.5$$

Where:

- α **and** β are the chosen angles.
- F is the fidelity after degradation.
- $\cos(\cdot)$ represents the correlation function.

This function behaves correctly because when the fidelity is zero, the probability becomes 0.5, meaning the output is completely random.

Mallory: The most appropriate way to model the presence of Mallory in a connection was to define links as either secure or insecure, and to execute different logic depending on the type of link.

When a link is defined as insecure, a measurement is performed and stored in the pair before the actual measurement. Then, in subsequent measurements, the basis calculation and entanglement probability are computed using Mallory's measurements instead of the actual client node measurements.

This result in a completely random outcome.

3.3 Obstacles

This section exposes some of the many obstacles found during the design and development process.

The first obstacle arose during the design phase. During the investigation, it was found that many simulators model fidelity degradation through multiple physical effects, such as fiber optic attenuation, quantum memory efficiency, and other channel imperfections. However, since this project focuses on the E91 protocol simulation and CHSH inequality validation, a highly detailed physical model of fidelity degradation was not required.

After extensive research, the paper [8] showed that after 100 km of fiber optic transmission, the entanglement fidelity decreases to approximately 86%, corresponding to a reduction of about 0.14% per kilometer. Therefore, a constant fidelity degradation rate of 0.14% per kilometer was chosen for the simulation.

During the development of the simulator, several obstacles were encountered, mainly due to Rust's programming rules.

In a simulation where events are scheduled based on runtime conditions, the system needs a way to execute functions dynamically. In Python, this can be achieved using reflection, where a dotted string path is resolved at runtime into a callable object .However, Rust is a statically compiled language and does not provide built-in runtime reflection of function names.

The solution was to create a registry that stored a function HashMap. This HashMap would map a EventName instance to its corresponding function stored in a Box pointer.

The last obstacle was performance. At the beginning, persistent storage was used as the storage for everything. This meant that not only nodes and links were stored, but also measurements and entangled pairs. As a result, a simulation with 10,000 qubits took more than 5 minutes to complete due to database acceses and locks between threds.

The solution was to move away from a shared database state between threads and instead adopt an in-memory HashMap-based state managed directly by the simulation loop.

4 EVALUATION

This section evaluates the correctness, security, and performance of the proposed system. The experiments are designed to verify that the protocol behaves as expected under both normal and adverse conditions, while also assessing its scalability and sensitivity to physical-layer parameters.

For each experiment, we define the simulation setup, the metrics collected, and the expected outcome. The primary metrics considered throughout this evaluation are the CHSH S parameter, key correctness, simulation execution time, and protocol termination behavior.

To evaluate the performance and correctness of the system, we will conduct the following tests:

- **1. Simulation run with two client nodes and an EPR source**: This is the simplest and most important test. For a successful run, the CHSH S parameter should be greater than 2, indicating a violation of the classical bound, and the keys generated by both nodes must be identical.
- **2. Simulation run over an insecure channel**: This test demonstrates the effects of operating the simulation over an insecure channel. For a successful test, the CHSH S parameter should fall below 2, indicating the absence of quantum correlations, and the protocol should terminate the key-generation process.
- **3. Relationship between CHSH values and distance**: This test examines the impact of distance on fidelity degradation and, consequently, on the CHSH value. Since fidelity degradation is expected to increase with distance, the measured CHSH value should decrease accordingly.
- **4. Impact of dark counts on simulation time**: This test investigates how detector dark counts affect detector cooldown periods and, consequently, the total time required to complete a simulation run. An increase in the dark count rate is expected to increase the overall simulation time.

4.1 Simulation run with two client nodes and an EPR source

This experiment validates the basic functionality of the system under normal operating conditions. The objective is to verify that entanglement distribution, CHSH-based security verification, and key generation are performed correctly.

The simulation consists of two client nodes, denoted as `src` and `dst`, connected to a shared EPR source through secure links. Each client is connected to the EPR node via a channel of 100 000 meters (100 km). Figure 4 illustrates the network topology used in this experiment.

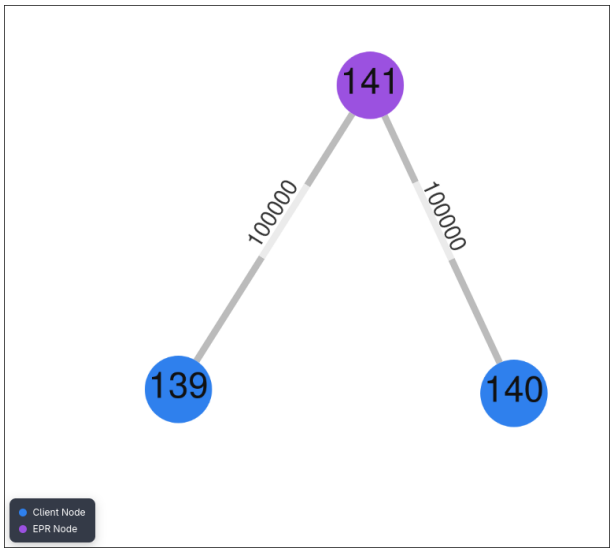

Figure 4: Diagram showing the used topology

For the protocol to be considered successful, the measured CHSH S parameter must be greater than 2, demonstrating the presence of quantum correlations, and both client nodes must generate identical secret keys.

In order to generate a final secret key of 1024 bits, the protocol must distribute at least 10,000 entangled pairs. Since the CHSH protocol employs three possible measurement bases per participant, only approximately $\frac{1}{9}$ of the distributed pairs are expected

to be measured using the same basis combination required for key generation.

As the CHSH S-paremeter is not deterministic, we will perform 10 runs and average the CHSH value accross all of them.

The results of the 10 runs are shown in Table 1

Table 1: CHSH values obtained over 10 simulation runs.

Run	CHSH Value
1	2.488
2	2.468
3	2.503
4	2.452
5	2.475
6	2.439
7	2.459
8	2.357
9	2.434
10	2.410
AVG	2.449

As the runs yield an average CHSH value of 2.449, this confirms that the correlations between the measurement bases in the test qubits are correct and demonstrates the presence of entanglement between the qubits. We now examine the generated keys.

For brevity, only the keys from a single run are shown, and they are presented in hexadecimal format.

The two keys generated were:

Source key: `f4aa5b5...cb6ba6`

Destination key: `f4aa5b5...cb6ba6`

The results confirm that the system operates correctly. All 10 runs produced a CHSH S value exceeding the classical bound of 2, with an average of 2.449. The deviation from the theoretical maximum is expected, as fidelity degradation over the 100 km channel introduces noise into the entangled pairs.In addition to this, the source and destination keys are identical across all runs, confirming that the sifting and error correction function as intended

4.2 Simulation run over an insecure channel

This experiment validates the basic functionality of insecure channels. The objective is to verify that CHSH-based security verification and mallory actuation are performed correctly.

The simulation consists of two client nodes, denoted as `src` and `dst`, connected to a shared EPR source through a secure and insecure link respectively. Each client is connected to the EPR node via a channel of 100 000 meters (100 km). Figure **??** illustrates the network topology used in this experiment.

For the protocol to be considered successful, the measured CHSH S parameter must be below 2, demonstrating the presence of a third person that breaks quantum correlations, and the simulation run must be stopped.

As the CHSH S-paremeter is not deterministic, we will perform 10 runs and average the CHSH value accross all of them.

The results of the 10 runs are shown in Table 2

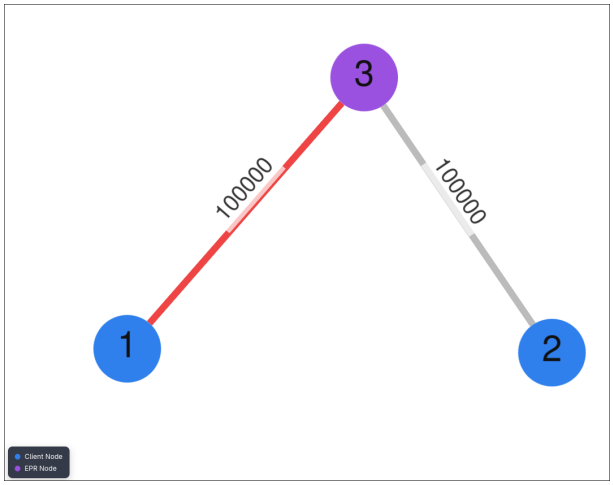

Figure 5: Diagram showing the used topology

Table 2: CHSH values obtained over 10 simulation runs in an insecure channel.

Run	CHSH Value
1	0.444
2	0.578
3	0.517
4	0.476
5	0.461
6	0.559
7	0.533
8	0.567
9	0.399
10	0.380
AVG	0.491

As shown in Table 2, the average CHSH value across all runs is 0.491. This significant reduction below the classical threshold of $S = 2$ indicates the absence of quantum correlations in the presence of an adversarial or insecure channel.

Additionally, the greater variance in the measurement outcomes suggests increased randomness, consistent with the disruption of entanglement correlations under insecure conditions.

In addition to this, we can see that the simulator displays the following error:

'Mallory was detected with CHSH 0.380 in process 0'

The results confirm that the system operates as intended in the presence of insecure channels. All 10 runs produced a CHSH S value below the classical bound of 2. As expected, all runs were halted due to the detection of Mallory.

4.3 Relationship Between CHSH Value and Distance

This experiment evaluates the behavior of the CHSH value when we alter the distance between communicating nodes. The expected

outcome is a decrease in the CHSH value as the distance increases, due to the progressive degradation of entanglement fidelity.

The simulation consists of two client nodes, denoted as `src` and `dst`, connected to a shared EPR source through a secure channel. Measurements will be performed at five different distances: 1 km, 50 km, 100 km, 200 km, and 400 km. At 1 km, fidelity degradation is expected to be negligible, and therefore the CHSH value should remain close to its theoretical maximum of approximately 2.8.

The experiment will be considered successful if the measured CHSH parameter, S, exhibits a decreasing trend as the distance between the nodes increases.

Since the CHSH parameter is not deterministic, two independent runs will be performed for each distance.

Table 3: CHSH values obtained over 10 simulations at different distances.

Run	1 km	50 km	100 km	200 km	400 km
1	2.841	2.573	2.543	2.001	1.159
2	2.778	2.641	2.493	2.057	1.149
AVG	2.810	2.607	2.518	2.029	1.154

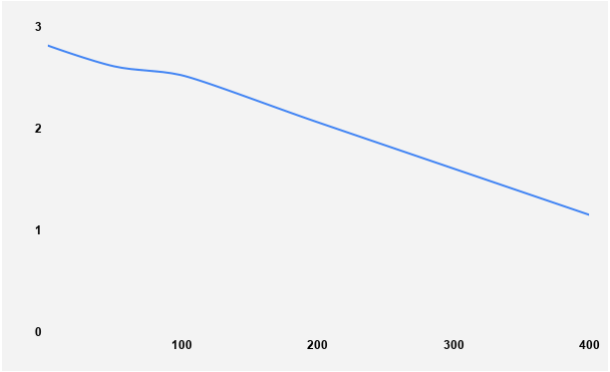

Figure 6: Diagram showing the relation of chsh and distance

The results confirm that the system behaves as expected when node distances vary. As the distnaec increases, the measured CHSH S value decreases, falling from an average of 2.810 at 1 km (fidelity degradation is negligible) to 1.154 at 400 km. This trend is consistent with the expected degradation of entanglement fidelity over longer distances. As expected, the CHSH value remains above the classical bound of 2 for shorter distances and eventually falls below this threshold at 400 km due to increased channel noise.

From Figure 7, we observe that the relationship between the CHSH value and distance is approximately linear. This is expected, as the fidelity degradation per km was simulated as a constant factor.

4.4 Impact of dark counts on simulation time

This experiment evaluates how detector dark counts affect the total execution time of the simulation. Dark counts introduce random detection events, which increase measurement time due to additional events that increase the detector cooldown.

The expected outcome is that higher dark count rates increase runtime, since more entangled-pair generation attempts are required to achieve the target key length under increased noise conditions.

Since the time to perform the run is non-deterministic, three independent runs will be performed for each dark count.

Table 4: Execution times over 3 runs with different count rates.

Run	10	100000	10000000
1	15.7 s	17.9 s	18.5 s
2	12.9 s	16.5 s	16.3 s
3	14.9 s	14.9 s	18.8 s
AVG	14.5 s	16.4 s	17.8 s

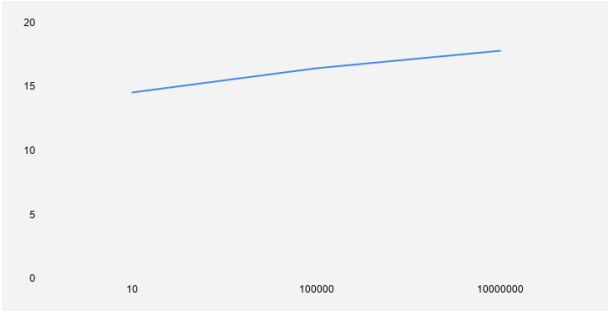

Figure 7: Diagram showing the relation of time and dark count rate

The results show that increasing dark count rates lead to longer simulation times. This happens because more events cause additional detector cooldowns n.

5 DISCUSSION

The results obtained confirm that the simulator behaves correctly with the theoretical predictions of CHSH and E91 protocol. In particular, the observed violation of the classical bound $S \leq 2$ in the secure channel experiments demonstrates that the simulator correctly reproduces non-classical correlations.

The average CHSH value of 2.449 in the two-node experiment indicates a strong, though not maximal, violation of the CHSH inequality. This reduction from the theoretical maximum is expected due to noise introduced by channel losses over the 100 km fibre link.

In contrast, the insecure channel experiment shows a CHSH value below the CHSH threshold. This indicates that the entanglement correlations are being destroyed in the presence of Mallory.

The detection and termination of the protocol in these cases demonstrates that CHSH violation is a reliable security indicator.

The relationship between CHSH value and distance supports the behavior of entangled qubits. As distance increases, fidelity degradation occurs, which in turn reduces the CHSH value. The approximately linear decrease observed in the results aligns with expectations as the fidelity degradation is simulated as as constant of 0.14%

Scalability results suggest that the system exhibits near-linear growth in execution time as the number of qubits increases. This is consistent with the expected complexity of entanglement distribution and measurement simulation, where each additional qubit pair introduces proportional computational overhead. However, this also implies that large-scale simulations (e.g., 10^5 qubits) may become computationally expensive, potentially limiting real-time applicability without further optimization or parallelization.

Several limitations should be noted. First, the noise and fidelity models used in the simulation are simplified and may not capture all real-world quantum channel effects such as time-varying decoherence or non-Markovian noise. Second, only a small number of runs were performed for statistical averaging, which limits confidence intervals on the CHSH estimates. Finally, the adversarial model is abstracted as a simple disturbance rather than a fully adaptive eavesdropper, which may overestimate detection robustness in practical deployments.

Overall, the results show that the system succesfully implements CHSH-based verification for E91 key distribution and demonstrates that physical behavior were simulated correctly.

6 THREATS TO VALIDITY

6.1 Internal Validity

Implementation errors and bugs in the simulator may pose a threat to internal validity. Although the results support the hypotheses and align with the expected behavior of the system, undetected issues in the implementation could have contributed to these outcomes. Therefore, even if the results appear consistent with theoretical expectations, the underlying mechanisms producing them may not be entirely correct.

The main area of concern is the implementation of Mallory. While the observed CHSH value is below the classical threshold, this alone does not guarantee that Mallory has been modeled accurately. In particular, any process introducing sufficient randomness into the measurements may lead to a CHSH value below 2, regardless of whether the attack is correctly implemented.

6.2 External Validity

Even though E91 QKD network topologies can be tested with this simulator, allowing the exploration of optimal distances and relevant system parameters, the simulator cannot be fully generalized to real-world scenarios for several reasons.

First, the scope of the simulator is limited to a single protocol, E91. As a result, other quantum key distribution protocols cannot be evaluated using the current framework. Future work will aim to extend the simulator to improve its flexibility.

In addition, due to time constraints and the scope of the project, physical effects such as fidelity degradation over distance are modeled in a simplified manner. Instead of simulating a combination of physical factors, fidelity is represented using a constant degradation factor. In real-world scenarios, however, fidelity is influenced by multiple variables, including fiber attenuation, environmental conditions, and hardware imperfections. Therefore, modeling it as a constant factor does not fully capture the complexity of real quantum communication systems.

6.3 Construct Validity

Construct validity of this simnulatopr is threatened by many assumptions made during the developemnt. Apart from the before mentioned fidelity degradation, using the CHSH value as the only metric for correctness does not take into account other metrics such as key rate.

6.4 Conclusion Validity

The conclusions drawn from the experimental results are supported by the observed data and are consistent across multiple simulation runs. The measured CHSH values, key agreement outcomes, and observed trends with respect to distance all align with the expected theoretical behavior of the E91 protocol.

7 RELATED WORK

A number of quantum network simulators have been developed to study entanglement distribution and quantum key distribution. These tools differ in abstraction level, ranging from circuit-based quantum simulators to discrete-event network simulators like this one.

In the following sections, we discuss SeQUeNCe, Qunetsim, QKDNetsim, SimulaQron, Qiskit, and NetSquid, focusing on their main design choices.

Sequence [12] is the most similar simulator to this work, as it served as the primary inspiration for its design. According to the authors, its main goal is to provide a framework suitable for simulating quantum network prototypes for both current and future technologies.

Although our simulator differs in scope, as this one is specifically designed for the E91 QKD protocol, the two systems share several architectural similarities. **Sequence** is an event-driven simulator that supports precise event scheduling with timestamp accuracy. It also follows a layered architecture with a clear separation of concerns.

However, **Sequence** extends this design with additional abstraction layers, such as dedicated network and photon layers. Furthermore, its fidelity modelling differs from ours: instead of applying a constant degradation factor per kilometer, it incorporates a combination of quantum memory fidelity degradation and additional physical parameters to more accurately represent realistic quantum network behavior.

Another example of a layered simulator is QKDNetSim [6]. This simulator provides multiple layers, such as the key management layer and the quantum layer. It was introduced in 2017 as the first simulation environment for quantum key distribution networks.

The main similarity with this simulator is that its primary focus is on quantum key distribution.

However, QKDNetSim differs in that it does not model quantum physical behavior. Instead, it simulates only the packet-based communication required for QKD between nodes. For this reason, QKDNetSim is built on top of ns-3, a discrete-event, classical network simulator.

QunetSim [2],similar to SeQUeNCe, it provides a flexible, event-driven framework in which users define network topologies and protocols programmatically rather than through a dedicated user interface. QunetSim also supports larger quantum networks through intermediary hosts and entanglement swapping, enabling the simulation of multi-hop entanglement distribution. Unlike this work, however, it is designed as a general-purpose quantum networking framework rather than a simulator focused specifically on the E91 QKD protocol.

8 CONCLUSION

In this thesis, we presented the design, implementation, and evaluation of a modular, high-performance discrete event-driven quantum network simulator written in Rust, made specifically for E91 Quantum Key Distribution (QKD) protocol. The development of this simulator addresses a key challenge in modern quantum communications: providing an accessible, robust tool to evaluate the relation quantum physical properties and classical post-processing.

The experimental results validate that our simulator successfully models distance-dependent entanglement degradation and non-deterministic detector properties, such as dark count rates driven by Poisson processes and detector cooldown periods. Through our test scenarios, the simulator demonstrated a strong violation of the classical CHSH inequality threshold ($S > 2$) over channel lengths up to 100 km, yielding an average Bell parameter $S = 2.449$. This reduction from the theoretical maximum correctly reflects the 0.14% fidelity degradation per kilometer modeled from real-world empirical data. Furthermore, the integration of an active third person (Mallory) confirmed the simulator's security robustness. The eavesdropper's interception attempts successfully destroys quantum correlations, collapsing the CHSH parameter well below the classical bound ($S = 0.491$) and accurately triggering immediate protocol termination.

The architecture's performance evaluation proved the scalability of our approach. By transitioning from a persistent database state to an in-memory HashMap-based architecture managed directly by the main execution loop, we mitigated thread-locking overhead and reduced simulation times exponentially, achieving a near-linear growth model suitable for large-scale runs up to 10^5 qubits.

8.1 Future Work

While the current implementation offers an end-to-end framework for evaluating the E91 protocol, several avenues remain for future research and enhancement:

- **Protocol Diversification:** The current architectural scope is restricted to the E91 protocol. Future iterations should introduce generalized abstraction layers to support other foundational protocols such as BB84, BBM92.
- **Advanced Physical Noise Frameworks:** To overcome the limitation of modeling channel noise as a constant linear degradation factor, the physical-layer simulation can be extended to incorporate complex time-varying decoherence, polarization mode dispersion, and non-Markovian noise models.

In summary, this work provides a reliable, secure, and computationally efficient foundation for quantum netw

REFERENCES

[1] Aliro Quantum. 2025. *Quantum Keys in Action: Using BBM92 for Secure Communication Today.* White Paper. Aliro Quantum. https://www.aliroquantum.com/hubfs/White%20Papers/Aliro%20White%20Paper%202025-06-19%20%20Quantum%20Keys%20in%20Action_%20Using%20BBM92%20for%20Secure%20Communication%20Today.pdf

[2] Stephen DiAdamo, Janis Nötzel, Benjamin Zanger, and Mehmet Mert Beşe. 2021. QuNetSim: A Software Framework for Quantum Networks. *IEEE Transactions on Quantum Engineering* 2 (2021), 2502512. https://doi.org/10.1109/TQE.2021.3092395

[3] Mikio Fujiwara, Ken-ichiro Yoshino, Yoshihiro Nambu, Taro Yamashita, Shigehito Miki, Hirotaka Terai, Zhen Wang, Morio Toyoshima, Akihisa Tomita, and Masahide Sasaki. 2015. Modified E91 Protocol Demonstration with Hybrid Entanglement Photon Source. *arXiv preprint arXiv:1501.05686* (Jan. 2015). arXiv:1501.05686 [quant-ph] https://arxiv.org/abs/1501.05686

[4] gonbvza. 2025. QKD Simulator: A Quantum Network Simulator Focused on the E91 QKD Protocol. https://github.com/gonbvza/qkd-simulator. Developed as a thesis project. Accessed: 2026-06-03.

[5] M. S. Guimarães, I. Roditi, and S. P. Sorella. 2024. Introduction to Bell's Inequality in Quantum Mechanics. arXiv:2409.07597 [quant-ph] https://arxiv.org/abs/2409.07597

[6] Miralem Mehic et al. 2022. QKDNetSim: Quantum Key Distribution Network Simulator — Model Library. https://www.qkdnetsim.info/models/build/html/qkdnetsim.html. Accessed: 2026-06-03.

[7] R. L. Rivest, A. Shamir, and L. Adleman. 1978. A Method for Obtaining Digital Signatures and Public-Key Cryptosystems. *Commun. ACM* 21, 2 (1978), 120–126.

[8] M. Sena et al. 2025. High-Fidelity Quantum Entanglement Distribution in Metropolitan Fiber Networks with Co-Propagating Classical Traffic. *Journal of Optical Communications and Networking* 17, 12 (2025), 1072–1081. https://doi.org/10.1364/JOCN.575396

[9] K. Muhammed Shafi, R. S. Gayatri, A. Padhye, and C. M. Chandrashekar. 2021. Bell-inequality in path-entangled single photon and purity of single photon state. https://doi.org/10.48550/arXiv.2112.05039 arXiv:2112.05039 [quant-ph]

[10] M. SujayKumar Reddy and B. Chandra Mohan. 2023. Comprehensive Analysis of BB84, A Quantum Key Distribution Protocol. *arXiv preprint arXiv:2312.05609* (Dec. 2023). arXiv:2312.05609 [quant-ph] https://arxiv.org/abs/2312.05609

[11] Kees van Hee, Kees van Berkel, and Jan de Graaf. 2026. A Probability Model for the Bell Experiment. *Quantum Reports* 8, 1 (2026), 16. https://doi.org/10.3390/quantum8010016

[12] Xiaoliang Wu, Alexander Kolar, Joaquin Chung, Dong Jin, Tian Zhong, Rajkumar Kettimuthu, and Martin Suchara. 2020. SeQUeNCe: A Customizable Discrete-Event Simulator of Quantum Networks. *arXiv preprint arXiv:2009.12000* (Sept. 2020). arXiv:2009.12000 [quant-ph] https://arxiv.org/abs/2009.12000